

Algorithm 1 — Boyer-Moore Voting (Non-Recursive)

Idea: A candidate that appears more than $n/2$ times can never be fully canceled by all other elements combined.

Pseudo-Code

Boyer-Moore Dominator — Pseudo-Code

```
FUNCTION findDominator(A[0..n-1]):  
  
    // Phase 1: Find Candidate  
    candidate <- A[0]  
    count      <- 1  
  
    FOR i <- 1 TO n-1 DO  
        IF count = 0 THEN  
            candidate <- A[i]  
            count      <- 1  
        ELSE IF A[i] = candidate THEN  
            count <- count + 1  
        ELSE  
            count <- count - 1  
        END IF  
    END FOR  
  
    // Phase 2: Verify Candidate  
    freq <- 0  
    FOR i <- 0 TO n-1 DO  
        IF A[i] = candidate THEN freq <- freq + 1  
    END FOR  
  
    IF freq > n/2 THEN  
        FOR i <- 0 TO n-1 DO  
            IF A[i] = candidate THEN RETURN i  
        END FOR  
    END IF  
  
    RETURN -1  
END FUNCTION
```

Complexity Analysis

Step-by-step count:

Phase	Operations	Explanation
Phase 1 — Voting	$n - 1$	One pass: compare + increment/decrement each element
Phase 2 — Verify	n	One pass: count occurrences of candidate
Phase 3 — Find idx	$\leq n$	One pass: return first matching index
Total	$3n$	Three linear passes $\rightarrow O(n)$

Time Complexity Calculation — Algorithm 1

$$\begin{aligned}T(n) &= (n-1) + n + n \\&= 3n - 1 \\&\sim 3n \\&= O(n)\end{aligned}$$

Space: only 2 variables used (candidate, count) $\rightarrow O(1)$

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Best Case: $O(n)$ | **Worst Case:** $O(n)$ | **Average:** $O(n)$

Algorithm 2 — Divide & Conquer (Recursive)

Idea: Split the array in half recursively until each part has one element. Merge back up by checking which candidate is dominant in the combined range.

Pseudo-Code

Divide & Conquer Dominator — Pseudo-Code

```
FUNCTION dominatorDC(A[], lo, hi):

    // Base case: single element
    IF lo = hi THEN RETURN A[lo]

    mid <- (lo + hi) / 2

    // Divide
    leftCandidate <- dominatorDC(A, lo, mid)
    rightCandidate <- dominatorDC(A, mid+1, hi)

    // Merge
    IF leftCandidate = rightCandidate THEN
        RETURN leftCandidate
    END IF

    size <- hi - lo + 1
    leftCount <- COUNT(A, lo, hi, leftCandidate)
    rightCount <- COUNT(A, lo, hi, rightCandidate)

    IF leftCount > size/2 THEN RETURN leftCandidate
    IF rightCount > size/2 THEN RETURN rightCandidate
    RETURN NO_CANDIDATE
END FUNCTION

FUNCTION COUNT(A[], lo, hi, val):
    freq <- 0
    FOR i <- lo TO hi DO
        IF A[i] = val THEN freq <- freq + 1
    RETURN freq
END FUNCTION

FUNCTION findDominator_DC(A[0..n-1]):
    candidate <- dominatorDC(A, 0, n-1)
    IF candidate = NO_CANDIDATE THEN RETURN -1
    FOR i <- 0 TO n-1 DO
        IF A[i] = candidate THEN RETURN i
    RETURN -1
END FUNCTION
```

Complexity Analysis

Recurrence Relation:

Recurrence Relation — Algorithm 2

$$T(n) = 2 * T(n/2) + n$$

Because:

- We split into 2 subproblems of size $n/2$
- COUNT() does n work at each merge level

Solving with Master Theorem:

Master Theorem Solution

$$T(n) = a * T(n/b) + f(n)$$

$a = 2$ (two recursive calls)

$b = 2$ (array split in half)

$f(n) = n$

$$\log_b(a) = \log_2(2) = 1$$

$f(n) = n = O(n^1) \rightarrow$ Case 2 of Master Theorem

Result: $T(n) = O(n \log n)$

Space: recursion depth = $\log_2(n) \rightarrow O(\log n)$

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$

Best Case: $O(n \log n)$ | **Worst Case:** $O(n \log n)$ | **Average:** $O(n \log n)$

Comparison

Criterion	Algorithm 1 — Boyer-Moore	Algorithm 2 — Divide & Conquer
Paradigm	Greedy / Linear Scan	Recursive Splitting
Time Complexity	$O(n)$	$O(n \log n)$
Space Complexity	$O(1)$	$O(\log n)$ stack
Passes	3 linear passes	$O(n \log n)$ comparisons
Code complexity	Simple	Moderate
Returns -1	Yes	Yes
Preferred	Yes	No (slower)

Conclusion:

Boyer-Moore is the optimal choice: $O(n)$ time and $O(1)$ space with simple logic.

Divide & Conquer is a valuable recursive exercise but is slower at $O(n \log n)$.